

✓ COURSE: Master calculus 1 using Python: derivatives and applications

SECTION: Applications

LECTURE: CodeChallenge: Gradient descent

TEACHER: Mike X Cohen, sincxpress.com

COURSE URL: [udemy.com/course/pycalc1_x/?couponCode=202307](https://www.udemy.com/course/pycalc1_x/?couponCode=202307)

Prepared by Prof Happy AI, 2026

Start coding or [generate](#) with AI.

```
# import all necessary modules
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt

import matplotlib_inline.backend_inline
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
```

Start coding or [generate](#) with AI.

✓ Mỗi quan hệ giữa một hàm số (function) và đạo hàm (derivative) của nó là nền tảng của giải tích và là chìa khóa để giải quyết các bài toán tối ưu hóa trong khoa học dữ liệu, vật lý và kinh tế. Dưới đây là các góc nhìn chuyên sâu về mối quan hệ này:

- Bản chất hình học: Giá trị và Độ dốc** Nếu hàm số $f(x)$ cho biết vị trí hoặc trạng thái tại một điểm, thì đạo hàm $f'(x)$ cho biết tốc độ thay đổi tức thời tại điểm đó. Hàm số $f(x)$: Đại diện cho một đường cong trên mặt phẳng tọa độ. Đạo hàm $f'(x)$: Tại mỗi điểm x , đạo hàm chính là hệ số góc (slope) của đường tiếp tuyến với đồ thị hàm số tại điểm đó.
- Ý nghĩa vật lý: Trạng thái và Sự biến thiên** Trong vật lý, mối quan hệ này thể hiện sự chuyển động: Nếu $s(t)$ là hàm quãng đường theo thời gian, thì $v(t) = s'(t)$ là vận tốc. Nếu $v(t)$ là vận tốc, thì $a(t) = v'(t) = s''(t)$ là gia tốc. Đạo hàm giúp chúng ta hiểu không chỉ cái gì đang xảy ra, mà là nó đang thay đổi nhanh hay chậm và theo hướng nào (tăng hay giảm).
- Phân tích hành vi hàm số qua đạo hàm** Đạo hàm là công cụ mạnh nhất để phân tích và hiểu đặc tính của hàm số mà không cần vẽ toàn bộ đồ thị. Tính đơn điệu: Nếu $f'(x) > 0$: Hàm số đang đồng biến (đi lên). Nếu $f'(x) < 0$: Hàm số đang nghịch biến (đi xuống). Điểm cực trị (Critical Points): Tại các điểm mà $f'(x) = 0$ (hoặc không xác định), hàm số có thể đạt cực đại hoặc cực tiểu. Đây là nơi "độ dốc" bằng phẳng trước khi đổi hướng. Độ cong (Đạo hàm cấp hai): $f''(x) > 0$: Đồ thị lõm xuống (concave up). $f''(x) < 0$: Đồ thị lõm lên (concave down).
- Ứng dụng trong Tối ưu hóa (Optimization)** Trong các ngành kỹ thuật và AI, mục tiêu thường là tìm giá trị nhỏ nhất của một hàm chi phí (Loss function). Chúng ta không thể "nhìn" thấy đáy của một hàm số nghìn chiều. Đạo hàm đóng vai trò như một chiếc la bàn: Nó chỉ ra hướng dốc nhất. Bằng cách đi ngược hướng của đạo hàm (Gradient Descent), chúng ta có thể tìm thấy điểm thấp nhất nơi đạo hàm bằng 0.
- Góc nhìn triết học: Cận cảnh và Toàn cảnh** Hàm số $f(x)$ là cái nhìn toàn cảnh (Global) về một hệ thống. Đạo hàm $f'(x)$ là cái nhìn cận cảnh (Local). Nó cho biết nếu ta thay đổi x một lượng cực nhỏ dx , thì $f(x)$ sẽ phản ứng như thế nào. Sự kết nối giữa chúng thông qua Định lý cơ bản của Giải tích cho thấy rằng tích phân (tổng hợp cái cục bộ) sẽ trả lại cho chúng ta hàm số ban đầu (cái toàn cục).

```
# function (as a function)
def fx(x):
    return 3*x**2 - 3*x + 4

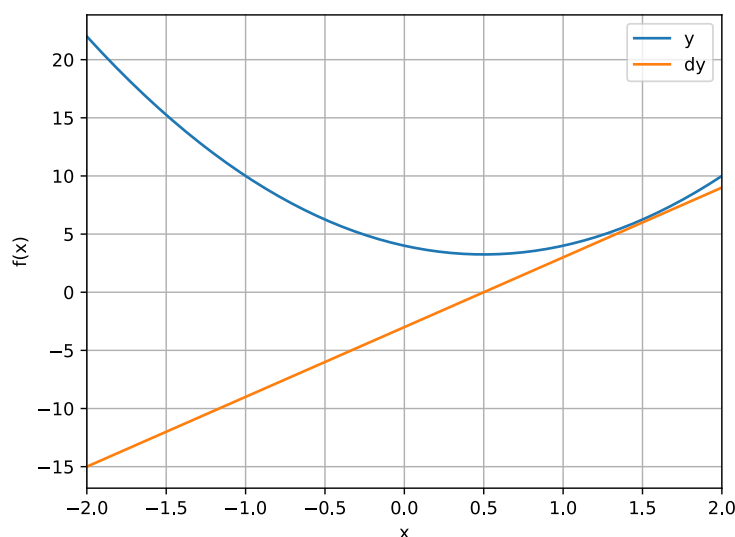
# derivative function
def deriv(x):
    return 6*x - 3
```

```
# plot the function and its derivative

# define a range for x
xx = np.linspace(-2,2,2001)

# plotting
plt.plot(xx,fx(xx), xx,deriv(xx))
```

```
plt.xlim(xx[[0,-1]])
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['y', 'dy'])
plt.show()
```



Start coding or [generate](#) with AI.

✧ Bài tập 2: Gradient descent

Gradient Descent (Cực tiểu hóa độ dốc) là một thuật toán tối ưu hóa nền tảng được sử dụng để tìm giá trị nhỏ nhất của một hàm số bằng cách di chuyển lặp đi lặp lại theo hướng dốc xuống mạnh nhất. Trong học máy (machine learning), đây là "động cơ" chính được dùng để cập nhật các tham số (trọng số và độ lệch) của mô hình nhằm giảm thiểu Hàm mất mát (sai số).

1. Ý tưởng cốt lõi: Hãy tưởng tượng bạn đang đứng trên đỉnh một dãy núi trong một màn sương mù dày đặc. Mục tiêu của bạn là xuống được điểm thấp nhất dưới thung lũng. Vì không thể nhìn thấy đáy, bạn cảm nhận độ dốc của mặt đất dưới chân mình. Bạn bước một bước nhỏ theo hướng mà mặt đất dốc xuống mạnh nhất. Bạn lặp lại quá trình này cho đến khi mặt đất trở nên bằng phẳng, dấu hiệu cho thấy bạn đã chạm đến điểm cực tiểu.

2. Cơ chế toán học: Để cập nhật một tham số θ , thuật toán tuân theo quy tắc cập nhật sau:

$$\theta_{next} = \theta_{current} - \eta \cdot \nabla J(\theta)$$

Trong đó: $J(\theta)$: Hàm mất mát (tương ứng với "ngọn núi"). $\nabla J(\theta)$: Gradient (hướng và độ lớn của độ dốc lớn nhất). η (Eta): Tốc độ học (Learning Rate). Đại lượng này quyết định kích thước của mỗi bước đi. Vai trò của Tốc độ học (Learning Rate): Tốc độ học là một siêu tham số (hyperparameter) cực kỳ quan trọng: Quá nhỏ: Thuật toán cuối cùng sẽ tìm thấy điểm cực tiểu, nhưng sẽ mất rất nhiều thời gian (chi phí tính toán cao). Quá lớn: Thuật toán có thể nhảy vượt qua điểm cực tiểu, dao động qua lại, hoặc thậm chí không bao giờ hội tụ được.

```
# random starting point
localmin = np.random.choice(xx,1)
print(f'Started at x = {localmin[0]:.3f}')

# learning parameters
learning_rate = .01
training_epochs = 100

# run through training
for i in range(training_epochs):
    grad = deriv(localmin)
    localmin = localmin - learning_rate*grad

print(f'Finished at x = {localmin[0]:.3f}')
```

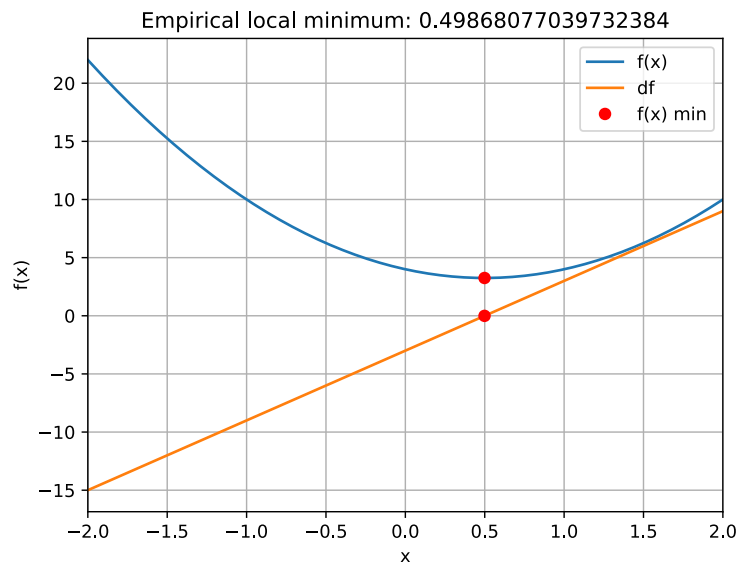
```
Started at x = -0.142
Finished at x = 0.499
```

```
# plot the results

plt.plot(xx,fx(xx), xx,deriv(xx))
plt.plot(localmin,deriv(localmin),'ro')
plt.plot(localmin,fx(localmin),'ro')

plt.xlim(xx[[0,-1]])
plt.grid()
```

```
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['f(x)', 'df', 'f(x) min'])
plt.title('Empirical local minimum: %s'%localmin[0])
plt.show()
```



Start coding or [generate](#) with AI.

✎ bài tập 3: Plot the descent

```
# Note: I don't discuss this in the video,
# but it's insightful to modify the parameters
# and observe the effects on the results. For example,
# fix the localmin and adjust the learning rate and epochs!
```

```
# fix the starting point
localmin = -1.5
```

```
# learning parameters
learning_rate = .05
training_epochs = 40
```

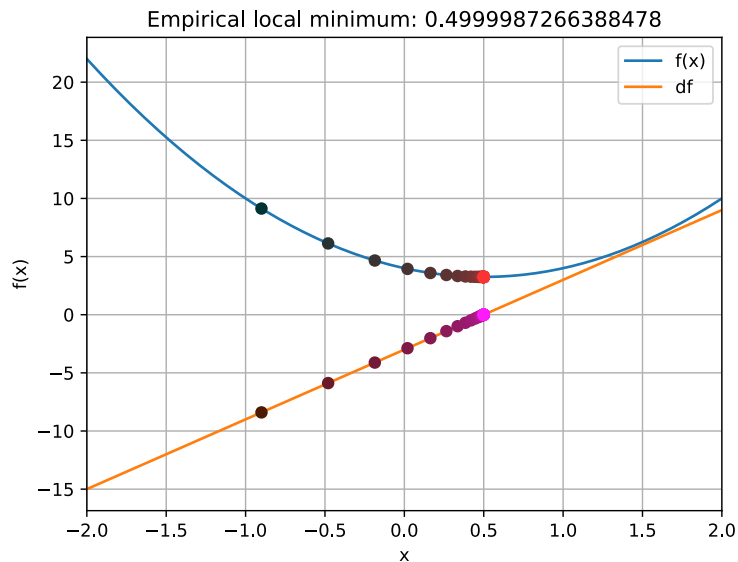
```
# run through training and store all the results
modelparams = np.zeros((training_epochs,2))
for i in range(training_epochs):
    grad = deriv(localmin)
    localmin = localmin - learning_rate*grad
    modelparams[i,0] = localmin
    modelparams[i,1] = grad
```

```
# plot the results
```

```
# plot the function
plt.plot(xx,fx(xx), xx,deriv(xx))
```

```
# plot the progression of points
for i,(xi,dx) in enumerate(modelparams):
    plt.plot(xi,fx(xi), 'o',color=[(i/training_epochs)**(1/2),.2,.2])
    plt.plot(xi,deriv(xi), 'o',color=[(i/training_epochs)**(1/2)*.7+.3,.1,(i/training_epochs)**(1/2)])
```

```
# adjust some plotting aspects
plt.xlim(xx[[0,-1]])
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['f(x)', 'df'])
plt.title('Empirical local minimum: %s'%modelparams[-1,0])
plt.show()
```



Start coding or [generate](#) with AI.

Exercise 4: Now with sympy

```
x = sym.var('x')

# define the function and its derivative
fx = sym.sin(x) * (x-2)**2
df = sym.diff(fx,x)

# quick plot
sym.plot(fx,(x,0,5*sym.pi));

# lambdify the functions (easier for plotting)
fx_lam = sym.lambdify(x,fx)
df_lam = sym.lambdify(x,df)

# a nicer-looking plot
xx = np.linspace(0,5*np.pi,2001)

_,axs = plt.subplots(2,1,figsize=(8,6))
axs[0].plot(xx,fx_lam(xx),linewidth=3)
axs[0].set_xlim(xx[[0,-1]])
axs[0].grid()
axs[0].set_title(f'f(x) = ${sym.latex(fx)}$')

axs[1].plot(xx,df_lam(xx),linewidth=3)
axs[1].plot(xx[[0,-1]],[0,0], 'k--')
axs[1].set_xlim(xx[[0,-1]])
axs[1].grid()
axs[1].set_title(f"f'(x) = ${sym.latex(df)}$")

plt.tight_layout()
plt.show()
```

Start coding or [generate](#) with AI.

Bài tập 5: Gradient descent in sympy

```
# random starting point in x domain
localmin = np.random.choice(xx,1)[0]

# learning parameters
learning_rate = .05
training_epochs = 40

# setup the plot with functions
plt.figure(figsize=(8,6))
plt.plot(xx,fx_lam(xx),label='f(x)')
```

```
plt.plot(xx,df_lam(xx),label="f'(x)")

# run through training and plot immediately
for i in range(training_epochs):

    # implement gradient descent
    grad = df.subs(x,localmin)
    localmin = localmin - learning_rate*grad

    # plot the current result
    plt.plot(localmin,fx.subs(x,localmin), 'o',color=[(i/training_epochs)**(1/2),.2,.2])
    plt.plot(localmin,df.subs(x,localmin), 'o',color=[(i/training_epochs)**(1/2)*.7+.3,.1,(i/training_epochs)**(1/2)])

plt.grid()
plt.xlabel('x')
plt.ylabel("f(x) or f'(x)")
plt.xlim(xx[[0,-1]])
plt.legend()
plt.title('Empirical local minimum: %s'%localmin)
plt.show()
```

Start coding or [generate](#) with AI.